
Guía de Actividades Práctico-Experimentales Nro. 005

1. Datos Generales

Asignatura	Estructura de datos
Ciclo	3 A
Unidad	2
Resultado de aprendizaje de la unidad	Aplica los métodos de ordenación y búsqueda en la resolución de problemas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Título de la Práctica	Ordenación básica en Java: Burbuja, Selección e Inserción
Nombre del Docente	Andrés Roberto Navas Castellanos
Fecha	Jueves 13 de noviembre viernes 14 de noviembre
Horario	07h30 – 10h30 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	5 horas
Estudiantes	- Ana Panamito - Fernando Patiño

Link Github

https://github.com/Fede-dev11/Sorting_Taller5.git

2. Objetivo(s) de la Práctica:

- Implementar y comparar tres algoritmos de ordenación in-place sobre arreglos pequeños, y validar su funcionamiento con trazas y casos de prueba reproducibles.

3. Materiales y reactivos:

- Guía de pruebas con datasets y salidas esperadas.

4. Equipos y herramientas

- JDK OpenJDK (obligatorio).
- IDE: Visual Studio Code (extensión "Extension Pack for Java") o IntelliJ IDEA Community.
- Sistema de control de versiones: Git; repositorio en GitHub.
- EVA/Moodle institucional: para entrega de evidencias.
- Herramientas de documentación: README Markdown, editor ofimático (Google Docs/LibreOffice/Word).

5. Procedimiento / Metodología

Enfoque metodológico: ABPr (Aprendizaje Basado en Proyectos). Inicio

- Presentación breve de objetivos y alcance (qué implementar y cómo verificar).
- Organización individual o en parejas; crear repo Git y rama de trabajo.
- Revisión de firmas y criterios de evaluación (rúbrica).
- Lineamientos de uso responsable de IA.

Desarrollo

- Implementar en este orden: Inserción, Selección, Burbuja (facilita comparar).
- **Estructura del proyecto**
 - Paquete `ed.u2.sorting`.
 - Clases sugeridas: `InsertionSort`, `SelectionSort`, `BubbleSort`, `SortingDemo`, `SortingUtils`.

- **Firmas estándar (Java)**

```
public final class InsertionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
public final class SelectionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
public final class BubbleSort {  
    public static void sort(int[] a) { /* ... */ }  
}
```

- Todas **in-place** (modifican el arreglo recibido).
 - Agregar sobrecarga opcional `sort(int[] a, boolean trace)` para imprimir trazas.
- **Inserción (recomendado empezar aquí)**
 - Bucle externo `i=1..n-1`; insertar `a[i]` en `0..i-1`.
 - **Trazas**: imprimir posiciones movidas y arreglo resultante por iteración.
- **Selección**
 - Para cada `i`, hallar **mínimo** en `i..n-1` y cambiar por `a[i]`.
 - Contabilizar **swaps** (pocos intercambios).
- **Burbuja**
 - Pasadas sucesivas; intercambiar adyacentes "mal ordenados".
 - Optimización: corte temprano si no hubo **swaps** en una pasada.
- **Casos bordes**
 - Arreglo vacío, tamaño 1, todos iguales, ya ordenado, orden inverso, con duplicados.
- **Trazas y validación**
 - Ejecutar `SortingDemo` con datasets de la sección 10.
 - Guardar capturas/salidas en TXT para subir al EVA.

Cierre

- Comparación cualitativa: cuándo usar cada algoritmo (datos pequeños, casi ordenados, número de swaps, legibilidad de traza).
- Revisión de evidencias y dudas frecuentes.
- Recordatorio de entregables y modo de verificación automática en EVA.

6. Resultados esperados:

- ZIP con src/ (clases de sort y SortingDemo), y README.md (1 pág.: decisiones, casos borde, cómo ejecutar).
- Evidencias:
- Trazas/outputs (.txt o capturas) de los datasets requeridos.

---Datasets y evidencias de código funcional---

```
-----MENU-----  
1. Burbuja  
2. Selection Sort  
3. Insertion Sort  
4. Salir  
Ingrese una opcion valida  
|
```

- Al seleccionar la opinión va pedir la cantidad de elementos q va tener el arreglo
- Y tenemos que ir agregando uno por uno.

```
Ingrese una opcion valida  
1  
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5  
Ingrese los 5 elementos (uno por uno):  
Valor[0]: 8  
Valor[1]: 3  
Valor[2]: 6  
Valor[3]: 3  
Valor[4]: 9  
Arreglo a ordenar  
8 3 6 3 9  
Pasada 1: 3 6 3 8 9  
Pasada 2: 3 3 6 8 9  
Pasada 3: 3 3 6 8 9  
No hay mas cambios que realizar  
  
--- RESULTADO FINAL ---  
  
Arreglo ordenado  
3 3 6 8 9
```

- Podemos ver en este las comparaciones que hace par el método selection

```

2
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5
Ingrese los 5 elementos (uno por uno):
Valor[0]: 8
Valor[1]: 3
Valor[2]: 6
Valor[3]: 3
Valor[4]: 9
Arreglo a ordenar
8 3 6 3 9
--- PASADA 1 ---Comparaciones
Minimo inicial posicion[0] = 8Posicion[1] = 3 es < valor posicion[0] = 8
SIMinimo encontrado posición 1 valor =3
Posicion[2] = 6 es < valor posicion[1] = 3
Posicion[3] = 3 es < valor posicion[1] = 3
Posicion[4] = 9 es < valor posicion[1] = 3
Arreglo después del intercambio: 3 8 6 3 9

--- PASADA 2 ---Comparaciones
Minimo inicial posicion[1] = 8Posicion[2] = 6 es < valor posicion[1] = 8
SIMinimo encontrado posición 2 valor =6
Posicion[3] = 3 es < valor posicion[2] = 6
SIMinimo encontrado posición 3 valor =3
Posicion[4] = 9 es < valor posicion[3] = 3
Arreglo después del intercambio: 3 3 6 8 9

--- PASADA 3 ---Comparaciones
Minimo inicial posicion[2] = 6Posicion[3] = 8 es < valor posicion[2] = 6
Posicion[4] = 9 es < valor posicion[2] = 6
Arreglo después del intercambio: 3 3 6 8 9

--- PASADA 4 ---Comparaciones
Minimo inicial posicion[3] = 8Posicion[4] = 9 es < valor posicion[3] = 8
Arreglo después del intercambio: 3 3 6 8 9

--- RESULTADO FINAL ---
Arreglo ordenado (selection)
3 3 6 8 9

```

- Podemos ver el método insertion

```

Ingrese una opcion valida
3
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5
Ingrese los 5 elementos (uno por uno):
Valor[0]: 8
Valor[1]: 3
Valor[2]: 6
Valor[3]: 3
Valor[4]: 9
Arreglo a ordenar
8 3 6 3 9
Paso 1 - Insertar: 3
    3 < 8 Izquierda 8
3 8 6 3 9
Paso 2 - Insertar: 6
    6 < 8 Izquierda 8
3 6 8 3 9
Paso 3 - Insertar: 3
    3 < 8 Izquierda 8
    3 < 6 Izquierda 6
3 3 6 8 9
Paso 4 - Insertar: 9
3 3 6 8 9
=== Parciales generados ===
Parcial: 3 8
Parcial: 3 6 8
Parcial: 3 3 6 8
Parcial: 3 3 6 8 9
--- RESULTADO FINAL ---
Arreglo ordenado (insertion)
3 3 6 8 9

```

- Segundo caso arreglo ordenado pero invertido

```

Ingrese una opcion valida
1
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5
Ingrese los 5 elementos (uno por uno):
Valor[0]: 5
Valor[1]: 4
Valor[2]: 3
Valor[3]: 2
Valor[4]: 1
Arreglo a ordenar
5 4 3 2 1
Pasada 1: 4 3 2 1 5
Pasada 2: 3 2 1 4 5
Pasada 3: 2 1 3 4 5
Pasada 4: 1 2 3 4 5
--- RESULTADO FINAL ---
Arreglo ordenado (Burbble)
1 2 3 4 5

```

Ingrese los 5 elementos (uno por uno):

Valor[0]: 5

Valor[1]: 4

Valor[2]: 3

Valor[3]: 2

Valor[4]: 1

Arreglo a ordenar

5 4 3 2 1

--- PASADA 1 ---Comparaciones

Mínimo inicial posicion[0] = 5 Posicion[1] = 4 es < valor posicion[0] = 5

SI Mínimo encontrado posición 1 valor =4

Posicion[2] = 3 es < valor posicion[1] = 4

SI Mínimo encontrado posición 2 valor =3

Posicion[3] = 2 es < valor posicion[2] = 3

SI Mínimo encontrado posición 3 valor =2

Posicion[4] = 1 es < valor posicion[3] = 2

SI Mínimo encontrado posición 4 valor =1

Arreglo después del intercambio: 1 4 3 2 5

--- PASADA 2 ---Comparaciones

Mínimo inicial posicion[1] = 4 Posicion[2] = 3 es < valor posicion[1] = 4

SI Mínimo encontrado posición 2 valor =3

Posicion[3] = 2 es < valor posicion[2] = 3

SI Mínimo encontrado posición 3 valor =2

Posicion[4] = 5 es < valor posicion[3] = 2

Arreglo después del intercambio: 1 2 3 4 5

--- PASADA 3 ---Comparaciones

Mínimo inicial posicion[2] = 3 Posicion[3] = 4 es < valor posicion[2] = 3

Posicion[4] = 5 es < valor posicion[2] = 3

Arreglo después del intercambio: 1 2 3 4 5

--- PASADA 4 ---Comparaciones

Mínimo inicial posicion[3] = 4 Posicion[4] = 5 es < valor posicion[3] = 4

Arreglo después del intercambio: 1 2 3 4 5

--- RESULTADO FINAL ---

Arreglo ordenado (selection)

1 2 3 4 5

- Insertion

Ingrese una opcion valida

3

Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5

Ingrese los 5 elementos (uno por uno):

Valor[0]: 5

Valor[1]: 4

Valor[2]: 3

Valor[3]: 2

Valor[4]: 1

Arreglo a ordenar

5 4 3 2 1

```

Paso 1 - Insertar: 4
  4 < 5 Izquierda 5
4 5 3 2 1
Paso 2 - Insertar: 3
  3 < 5 Izquierda 5
  3 < 4 Izquierda 4
3 4 5 2 1
Paso 3 - Insertar: 2
  2 < 5 Izquierda 5
  2 < 4 Izquierda 4
  2 < 3 Izquierda 3
2 3 4 5 1
Paso 4 - Insertar: 1
  1 < 5 Izquierda 5
  1 < 4 Izquierda 4
  1 < 3 Izquierda 3
  1 < 2 Izquierda 2
1 2 3 4 5
=== Parciales generados ===
Parcial: 4 5
Parcial: 3 4 5
Parcial: 2 3 4 5
Parcial: 1 2 3 4 5
--- RESULTADO FINAL ---
Arreglo ordenado (insertion)
1 2 3 4 5

```

- Tercer caso valores ya ordenados

```

Ingresar una opcion valida
1
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 5
Ingrese los 5 elementos (uno por uno):
Valor[0]: 1
Valor[1]: 2
Valor[2]: 3
Valor[3]: 4
Valor[4]: 5
Arreglo a ordenar
1 2 3 4 5
Pasada 1: 1 2 3 4 5
No hay mas cambios que realizar
--- RESULTADO FINAL ---
Arreglo ordenado (Burbble)
1 2 3 4 5

```

```

Valor[4]: 5
Arreglo a ordenar
1 2 3 4 5
--- PASADA 1 ---Comparaciones
Mínimo inicial posicion[0] = 1
Posicion[1] = 2 es < valor posicion[0] = 1
Posicion[2] = 3 es < valor posicion[0] = 1
Posicion[3] = 4 es < valor posicion[0] = 1
Posicion[4] = 5 es < valor posicion[0] = 1
Arreglo después del intercambio: 1 2 3 4 5

--- PASADA 2 ---Comparaciones
Mínimo inicial posicion[1] = 2
Posicion[2] = 3 es < valor posicion[1] = 2
Posicion[3] = 4 es < valor posicion[1] = 2
Posicion[4] = 5 es < valor posicion[1] = 2
Arreglo después del intercambio: 1 2 3 4 5

--- PASADA 3 ---Comparaciones
Mínimo inicial posicion[2] = 3
Posicion[3] = 4 es < valor posicion[2] = 3
Posicion[4] = 5 es < valor posicion[2] = 3
Arreglo después del intercambio: 1 2 3 4 5

--- PASADA 4 ---Comparaciones
Mínimo inicial posicion[3] = 4
Posicion[4] = 5 es < valor posicion[3] = 4
Arreglo después del intercambio: 1 2 3 4 5

--- RESULTADO FINAL ---
Arreglo ordenado (selection)
1 2 3 4 5

```

```

Arreglo a ordenar
1 2 3 4 5
Paso 1 - Insertar: 2
1 2 3 4 5
Paso 2 - Insertar: 3
1 2 3 4 5
Paso 3 - Insertar: 4
1 2 3 4 5
Paso 4 - Insertar: 5
1 2 3 4 5
=== Parciales generados ===
Parcial: 1 2
Parcial: 1 2 3
Parcial: 1 2 3 4
Parcial: 1 2 3 4 5
--- RESULTADO FINAL ---
Arreglo ordenado (insertion)
1 2 3 4 5

```

- Caso con valores duplicados 2 2 2 2


```

Ingrese una opcion valida
1
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 4
Ingrese los 4 elementos (uno por uno):
Valor[0]: 2
Valor[1]: 2
Valor[2]: 2
Valor[3]: 2
Arreglo a ordenar
2 2 2 2
El arreglo ya está ordenado (Burbuja)

Ingrese una opcion valida
1
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 4
Ingrese los 4 elementos (uno por uno):
Valor[0]: 2
Valor[1]: 2
Valor[2]: 1
Valor[3]: 2
Arreglo a ordenar
2 2 1 2
Pasada 1: 2 1 2 2
Pasada 2: 1 2 2 2
Pasada 3: 1 2 2 2
No hay mas cambios que realizar
--- RESULTADO FINAL ---
Arreglo ordenado (Burbble)
1 2 2 2

```

- Selection

```

Ingrese una opcion valida
2
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 4
Ingrese los 4 elementos (uno por uno):
Valor[0]: 2
Valor[1]: 2
Valor[2]: 2
Valor[3]: 2
Arreglo a ordenar
2 2 2 2
El arreglo ya está ordenado (selection)
-----
Arreglo a ordenar
2 2 1 2
--- PASADA 1 ---Comparaciones
Minimo inicial posicion[0] = 2
Posicion[1] = 2 es < valor posicion[0] = 2
Posicion[2] = 1 es < valor posicion[0] = 2
SI Minimo encontrado posición 2 valor =1
Posicion[3] = 2 es < valor posicion[2] = 1
Arreglo después del intercambio: 1 2 2 2

```

```

--- PASADA 2 ---Comparaciones
Minimo inicial posicion[1] = 2
Posicion[2] = 2 es < valor posicion[1] = 2
Posicion[3] = 2 es < valor posicion[1] = 2
Arreglo después del intercambio: 1 2 2 2

--- PASADA 3 ---Comparaciones
Minimo inicial posicion[2] = 2
Posicion[3] = 2 es < valor posicion[2] = 2
Arreglo después del intercambio: 1 2 2 2

--- RESULTADO FINAL ---
Arreglo ordenado (selection)
1 2 2 2

```

- Insertion

```

Ingrese una opcion valida
3
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 : 4
Ingrese los 4 elementos (uno por uno):
Valor[0]: 2
Valor[1]: 2
Valor[2]: 2
Valor[3]: 2
Arreglo a ordenar
2 2 2 2
El arreglo ya está ordenado (insertion)

Arreglo a ordenar
2 2 1 2
Paso 1 - Insertar: 2
2 2 1 2
Paso 2 - Insertar: 1
  1 < 2 Izquierda 2
  1 < 2 Izquierda 2
1 2 2 2
Paso 3 - Insertar: 2
1 2 2 2
=== Parciales generados ===
Parcial: 2 2
Parcial: 1 2 2
Parcial: 1 2 2 2
--- RESULTADO FINAL ---
Arreglo ordenado (insertion)
1 2 2 2

```

- Prueba con valores pequeños y grandes
 - Burbble

```

4. Salida
Ingrese una opcion valida
1
Ingrese la cantidad de elementos del arreglo debe ser mayor o igual a 0 :
4
Ingrese los 4 elementos (uno por uno):
Valor[0]: 9
Valor[1]: 1
Valor[2]: 8
Valor[3]: 2
Arreglo a ordenar
9 1 8 2
Pasada 1: 1 8 2 9
Pasada 2: 1 2 8 9
Pasada 3: 1 2 8 9
No hay mas cambios que realizar
--- RESULTADO FINAL ---
Arreglo ordenado (Burbble)
1 2 8 9

```

- Selection

```

Arreglo a ordenar
9 1 8 2
--- PASADA 1 ---Comparaciones
Minimo inicial posicion[0] = 9
Posicion[1] = 1 es < valor posicion[0] = 9
SIMinimo encontrado posición 1 valor =1
Posicion[2] = 8 es < valor posicion[1] = 1
Posicion[3] = 2 es < valor posicion[1] = 1
Arreglo después del intercambio: 1 9 8 2

--- PASADA 2 ---Comparaciones
Minimo inicial posicion[1] = 9
Posicion[2] = 8 es < valor posicion[1] = 9
SIMinimo encontrado posición 2 valor =8
Posicion[3] = 2 es < valor posicion[2] = 8
SIMinimo encontrado posición 3 valor =2
Arreglo después del intercambio: 1 2 8 9

--- PASADA 3 ---Comparaciones
Minimo inicial posicion[2] = 8
Posicion[3] = 9 es < valor posicion[2] = 8
Arreglo después del intercambio: 1 2 8 9

--- RESULTADO FINAL ---
Arreglo ordenado (selection)
1 2 8 9

```

- Insertion

```

Arreglo a ordenar
9 1 8 2
Paso 1 - Insertar: 1
  1 < 9 Izquierda 9
1 9 8 2
Paso 2 - Insertar: 8
  8 < 9 Izquierda 9
1 8 9 2
Paso 3 - Insertar: 2
  2 < 9 Izquierda 9
  2 < 8 Izquierda 8
1 2 8 9
=== Parciales generados ===
Parcial: 1 9
Parcial: 1 8 9
Parcial: 1 2 8 9
--- RESULTADO FINAL ---
Arreglo ordenado (insertion)
1 2 8 9

```

- Tabla breve comparando recuentos de movimientos/intercambios observados (cualitativo).

Algoritmo	Mecanismo Principal	Recuento de movimientos/intercambios observados (cualitativo).
InsertionSort	Inserta el elemento actual en su lugar correcto en la parte ya ordenada.	Muchos Movimientos, Pocos Intercambios. Mueve elementos a la derecha para crear un hueco, lo que requiere muchas asignaciones (movimientos), pero los <i>swaps</i> completos son raros.
SelectionSort	Encuentra el elemento mínimo en la sublista no ordenada y lo intercambia por el elemento en la posición actual <i>i</i> .	Pocos Intercambios (Swaps). Realiza solo un <i>swap</i> por pasada (máximo N-1 swaps totales), independientemente de la entrada.
BubbleSort	Intercambia elementos adyacentes si están en el orden incorrecto, "burbujeando" el mayor hacia el final.	Muchos Intercambios Adyacentes. Realiza muchos <i>swaps</i> adyacentes para mover un elemento a su posición, especialmente en el caso inverso.

7. Preguntas de Control:

Datasets y salidas esperadas:

- A = [8, 3, 6, 3, 9]
- B = [5, 4, 3, 2, 1] (inverso)
- C = [1, 2, 3, 4, 5] (ya ordenado)
- D = [2, 2, 2, 2] (duplicados)
- E = [9, 1, 8, 2]

Orden final esperado (para los tres algoritmos)

- A → [3, 3, 6, 8, 9]
- B → [1, 2, 3, 4, 5]
- C → [1, 2, 3, 4, 5]
- D → [2, 2, 2, 2]
- E → [1, 2, 8, 9]

Preguntas de control:

- **¿Por qué la Inserción es preferible con datos casi ordenados?**

Porque como tal este método de ordenamiento lo que hace es que se toma cada elemento y se lo mueve hacia la izquierda mientras sea menor que el anterior.

Si el arreglo ya está casi ordenado, cada elemento necesita muy pocos movimientos, a veces ninguno. Lo que ayuda a que reduzca el trabajo. PO lo que podemos decir que se aproxima $O(n)$.

- **¿Qué propiedad hace que la Selección use pocos swaps? ¿Qué compromisos tiene?**

Es la clave de Selección ya que nos dice que solo realiza un swap por cada pasada, debido a que en cada iteración busca el mínimo del subarreglo restante. Por lo cual solo al final de esa búsqueda intercambia ese mínimo con la posición actual. Por lo que el algoritmo siempre hace $n-1$ swaps, independientemente del estado inicial en el que esté el arreglo.

Esta ventaja tiene compromisos importantes. En primer lugar, al realizar una gran cantidad de comparaciones, pues siempre examina todo el subarreglo restante para hallar el mínimo, manteniendo una complejidad de $O(n^2)$ incluso cuando los datos ya están ordenados. Además, no es adaptativo por lo que no aprovecha el orden previo y su tiempo de ejecución no mejora en casos favorables, a diferencia de Inserción. Tampoco es estable, ya que al mover el mínimo hacia el frente puede alterar la posición relativa de elementos iguales. Y, su elevado número de comparaciones lo vuelve poco competitivo en arreglos grandes frente a algoritmos más eficientes.

- **¿Cómo implementarías el corte temprano en Burbuja y qué caso reduce significativamente?**

En caso donde el corte temprano en Burbuja se implemente detectando si, durante una pasada completa, no se realizó ningún intercambio. Si no hubo cambios, significa que el arreglo ya está ordenado y el algoritmo puede detenerse inmediatamente.

Lo cual reduce de forma notable el caso de datos ya ordenados o casi ordenados. Si no existe un corte temprano el Burbuja va realizar todas las pasadas, aunque no haya nada que ordenar. Y al tener un corte temprano, la complejidad se acerca a $O(n)$ en este caso, en lugar de $O(n^2)$.

- **¿Cuál(es) de los tres puede(n) ser estable y en qué condiciones?**

Burbuja

Puede ser estable siempre que los intercambios se hagan estrictamente cuando $a[j] > a[j+1]$. Ya que, en ese caso, dos elementos iguales nunca se cruzan y su orden relativo se conserva.

Insertión

Es estable. Mientras en la comparación se usa la condición $>$ para desplazar elementos y no $\geq$, ya que como tal los iguales no se mueven entre sí lo que nos va ayudar a conservar el orden original.

Selección

No es estable en su forma clásica ya que el intercambio del mínimo encontrado con la posición inicial puede mover un elemento "igual" desde más lejos y romper el orden relativo. Por lo que este puede llegar a ser estable si se modifica su implementación para que, en vez de intercambiar, se "inserte" el mínimo manteniendo el resto desplazado; esto cambia su comportamiento interno.

Menciona dos casos bordes que deben probarse siempre.

- Arreglo vacío o de tamaño 1: ya que nos va permitir verificar que el algoritmo no falla con tamaños mínimos y que respeta correctamente condiciones de límite.
- Arreglo ya ordenado: Sirve para comprobar si el algoritmo maneja correctamente el mejor caso posible y, si es adaptativo, confirmar que aprovecha ese orden previo.
- Arreglo con datos no uniformes: Esto nos va ayudar a verificar que no hay desbordamientos o comportamientos inesperados en comparaciones.

8. Evaluación

Criterio	4 – Excelente	3 – Bueno	2 – Básico	1 – Insuficiente	Pts
Implementación InsertionSort	Correcta; traza clara; maneja casos bordes	Detalles menores	Parcial	No funcional	3.0
Implementación SelectionSort	Correcta; contabiliza swaps	Detalles menores	Parcial	No funcional	2.5
Implementación BubbleSort	Correcta; corte temprano implementado	Detalles menores	Parcial	No funcional	2.5
Evidencias (README + salidas)	Completas, reproducibles	Aceptables	Escasas	Inexistentes	1.0
Calidad de código	Nombres/estructura/comentarios adecuados	Aceptable	Pobre	Deficiente	1.0

9. Bibliografía

[1] OpenDSA Project, "Sorting and Searching Modules," Virginia Tech, 2021–2024 (REA con visualizaciones y ejercicios).

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] Oracle, "Java SE 17–21 Documentation: Arrays, Collections, and I/O (java.nio.file)." 2021–2025.

[4] T. Roughgarden, Algorithms Illuminated (Part 1), Soundlikeyourself Publishing, 2019–2020 (con recursos online actualizados).

10. Elaboración y Aprobación

Elaborado por	Andrés R Navas Castellanos Docente	 Firmado electrónicamente por: ANDRES ROBERTO NAVAS CASTELLANOS Validar únicamente con FirmaBC
Revisado por Solo si es realizado en laboratorios	Luis Sinche Técnico Docente	No Aplica
Aprobado por	Edison L Coronel Romero Director de Carrera	 Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMERO Validar únicamente con FirmaBC